FilaFácil Logística

Python puro explicado para quem vem do Laravel

Cada conceito do projeto **simple-software-system** comparado com o que você já conhece de Laravel, Artisan, Collections, Storage e Log.

Laravel / PHP	Este projeto / Python

Legenda de cores usada em todo o guia: vermelho = lado Laravel, azul = lado Python.

Este projeto não usa banco de dados nem framework. Ele guarda tudo em um arquivo JSON e usa só a biblioteca padrão do Python. Para a versão com SQLite e SQLAlchemy, veja o outro guia em `docs/`.

1. A ideia central

Imagine um comando Artisan interativo, daqueles que usam **$this->ask()**, **$this->choice()** e **$this->table()**, mas que, em vez de gravar em MySQL, lê e escreve um arquivo `pickups.json` (como se usasse `Storage::put()`). Não há rotas, HTTP, Eloquent nem migrations. É isso que este projeto é.

O FilaFácil cadastra, lista, filtra, atualiza e resume **coletas** (pickups) de uma empresa de logística, tudo em um menu no terminal. Os dados ficam numa lista de dicionários em memória, e essa lista é gravada no JSON a cada alteração.

1.1 Mapa de arquivos

Laravel / PHP	Este projeto / Python
artisan + o método **handle()** do comando	`main.py`: ponto de entrada e menu em loop
Chamadas de **$this->ask/choice/table/info** do Artisan	`cli.py`: toda entrada e saída no terminal
Classe **Service** / Actions (regras de negócio)	`pickups.py`: regras, sem `input()` nem `print()`
Repository / **Storage::disk()->get/put**	`storage.py`: único módulo que toca no arquivo
config/*.php ou um **Enum** com **label()**	`constants.py`: prioridades, status e rótulos em português
Tabela **pickups** (ou um arquivo em **storage/app**)	`pickups.json`: criado sozinho no primeiro cadastro
storage/logs/laravel.log	`filafacil.log`: logs técnicos, nunca no terminal
.gitignore padrão do Laravel	`.gitignore`: ignora `pickups.json`, o log e `__pycache__/`
composer.json + **vendor/**	Nada: usa só a biblioteca padrão (`json`, `pathlib`, `logging`, `functools`)
README.md	`README.md`: como rodar e como usar

> **Resumo da analogia**
> O **cli.py** é o equivalente das chamadas de entrada e saída do Artisan. O **pickups.py** é a camada de Service: recebe dados, devolve dados, e não sabe que existe um terminal. O **storage.py** é o Repository. O **main.py** é o controller que liga tudo.

2. Quem chama quem

Uma ação desce do menu até o arquivo e a resposta volta pelo mesmo caminho. A regra mais importante do projeto: **cada arquivo tem uma única responsabilidade**.

```
main.py  --- usa --->  cli.py        (pergunta e mostra)
   |
   +------ usa --->  pickups.py    (decide: validar, filtrar, somar)
   |                    ^
   +------ usa --->  storage.py    (lê e grava o JSON)
                        |
                        +--- usa --> pickups.is_valid_pickup (valida o que veio do arquivo)

constants.py é importado por todos (listas de valores e rótulos)
```

2.1 Exemplo: “Cadastrar coleta” passo a passo

#	Quem	O que acontece	Equivalente Laravel
1	`main.py`	O usuário digita `1`. O `if/elif` chama `create_pickup(pickups)`.	O roteador chama o método do controller
2	`cli.py`	`read_text`, `read_int` e `read_option` perguntam até a resposta ser válida.	**$this->ask()** + validação repetindo a pergunta
3	`pickups.py`	`register_pickup` valida de novo, monta o dict e devolve uma **nova** lista.	**$service->create($data)**
4	`storage.py`	`save_pickups` grava a lista inteira no JSON.	**$repository->save()** / **Storage::put()**
5	`main.py`	Registra no log e mostra “Coleta #1 cadastrada”.	**Log::info()** + **$this->info()**

> **Por que validar duas vezes?**
> O `cli.py` valida para dar uma boa experiência (pergunta de novo, sem erro). O `pickups.py` valida de novo porque a regra de negócio **não pode confiar** em quem a chama. No Laravel é a mesma ideia de **FormRequest** na borda e regras no Service/Model por segurança.

3. Python para quem sabe PHP: o básico

A lógica é a mesma, o que muda é a sintaxe. Esta tabela cobre tudo que aparece no projeto.

Laravel / PHP	Este projeto / Python
$name = 'Ana'; (com `$` e `;`)	`name = "Ana"` (sem `$`, sem `;`)
Chaves `{ }` delimitam blocos	A **indentação** (4 espaços) delimita blocos, e termina com `:`
Array associativo `['id' => 1]`	**dict**: `{"id": 1}`
Array indexado `[1, 2, 3]`	**list**: `[1, 2, 3]`. E **tuple** `(1, 2)` (imutável, usada em `PRIORITIES`)
`null`	`None` (compara-se com `is None`)
`function foo(array $a): int`	`def foo(a: list[dict]) -> int:` (type hints opcionais)
`fn($p) => $p['id']`	`lambda p: p["id"]` (função anônima de uma linha)
String `"Olá {$name}"`	f-string: `f"Olá {name}"`
`in_array($x, $lista)`	`x in lista` (e `x not in lista`)
`throw new InvalidArgumentException('...')`	`raise ValueError("...")`
`try { } catch (Exception $e) { }`	`try:` ... `except ValueError as error:`
`use App\Services\PickupService;`	`from pickups import register_pickup`
`array_merge($a, ['k' => 'v'])` / `[...$a, $b]`	`{**a, "k": "v"}` (dict) e `[*lista, novo]` (list)
PHPDoc `/** ... */`	**docstring**: texto entre `"""` logo abaixo do `def`
Nomes em camelCase (`savePickup`)	Nomes em **snake_case** (`save_pickups`) e constantes em MAIÚSCULAS

Laravel / PHP	Este projeto / Python
Arquivo `index.php` executado direto	`if __name__ == "__main__": main()`: só roda `main()` se o arquivo for executado, não se for importado

Pegadinha nº 1: referência x cópia
No PHP, atribuir um array **copia** o valor. No Python, atribuir uma lista ou dict **aponta para o mesmo objeto**. Por isso este projeto nunca altera a lista original: `register_pickup` e `update_status` devolvem uma **lista nova**, e o `main.py` guarda o resultado com `pickups = ...`. Esquecer essa reatribuição é o erro mais comum (a tela não muda).

Pegadinha nº 2: o que é "falso"
Em Python, texto vazio `""`, `0`, `[]` e `None` são falsos. Diferente do PHP, a string `"0"` é **verdadeira**. Por isso `if value:` em `read_text` funciona para detectar texto vazio.

4. Arquivo por arquivo

4.1 constants.py: o "Enum" do projeto

Aqui ficam os valores aceitos e as traduções. O arquivo guarda inglês (`high`, `in_transit`) e o menu mostra português (`Alta`, `Em rota`), o mesmo papel de um Enum com método **label()**.

Laravel / PHP

```
enum PickupStatus: string {
  case Pending = 'pending';
  case InTransit = 'in_transit';
  case Completed = 'completed';

  public function label(): string {
    return match($this) {
      self::Pending => 'Pendente',
      self::InTransit => 'Em rota',
      self::Completed => 'Concluída',
    };
  }
}
```

Este projeto / Python

```
STATUSES = (
    "pending", "in_transit", "completed"
)
INITIAL_STATUS = "pending"

STATUS_LABELS = {
    "pending": "Pendente",
    "in_transit": "Em rota",
    "completed": "Concluída",
}
```

O Python tem `enum`, mas aqui bastou uma **tupla** para validar (`status in STATUSES`) e um **dict** para traduzir (`STATUS_LABELS[status]`).

4.2 cli.py: o "Artisan" do projeto

Todas as perguntas ao usuário ficam aqui. Cada função repete a pergunta até receber algo válido. Isso substitui o "redirect back with errors" do Laravel: no terminal não há redirecionamento, então o **loop** faz esse papel.

Laravel / PHP

```
$name = $this->ask('Nome do cliente');
// validação: required|string
// se falhar: mostra erro e pergunta
// de novo
```

Este projeto / Python

```
def read_text(prompt: str) -> str:
    while True:
        value = input(f"{prompt}: ").strip()
        if value:
            return value
        print_message(
            "O campo não pode ficar vazio.")
```

`input()` lê uma linha do teclado (como `$this->ask()`). `.strip()` tira os espaços das pontas (como `trim()`). `while True` é um loop infinito que só sai com `return`.

Laravel / PHP	Este projeto / Python

`$this->table(`
`  ['ID', 'Cliente', 'Endereço'],`
`  $pickups->map(fn($p) => [`
`  $p['id'], $p['customer_name'],`
`  $p['address'],`
`  ])`
`);`
	`def print_table(pickups: list[dict]) -> None:`
	`  if not pickups:`
	`  print_message(`
	`  "Nenhuma coleta encontrada")`
	`  return`
	`  header = ROW_FORMAT.format(*HEADERS)`
	`  print(header)`
	`  print("-" * len(header))`
	`  rows = map(to_display_row, pickups)`
	`  print("\n".join(rows))`

`map(to_display_row, pickups)` transforma cada coleta em uma linha de texto, e `"\n".join(...)` junta tudo (como `implode`). `ROW_FORMAT` usa larguras fixas para alinhar as colunas.

4.3 pickups.py: a camada de Service

São **funções puras**: recebem a lista de coletas e devolvem um resultado, sem `print()`, sem `input()` e sem tocar em arquivo. É o que torna fácil testar e reaproveitar. Como não há classes aqui, é um módulo de funções, e não um **PickupService** com métodos.

Laravel / PHP	Este projeto / Python

`public function register($pickups, $data) {`
`  $this->validate($data);`
`  $new = [`
`  'id' => $this->nextId($pickups),`
`  'status' => 'pending',`
`  ...$data,`
`  ];`
`  return [...$pickups, $new];`
`}`
	`def register_pickup(`
	`  pickups, customer_name, address,`
	`  package_count, priority):`
	`  validate_pickup_data(`
	`  customer_name, address,`
	`  package_count, priority)`
	`  new_pickup = {`
	`  "id": next_id(pickups),`
	`  "customer_name": customer_name.strip(),`
	`  ...`
	`  "status": INITIAL_STATUS,`
	`  }`
	`  return [*pickups, new_pickup]`

Repare no `[*pickups, new_pickup]`: cria uma **lista nova** com tudo da antiga mais o novo item, igual ao spread `[...$pickups, $new]` do PHP 8.

Laravel / PHP	Este projeto / Python

`if (! in_array($status, self::STATUSES)) {`
`  throw new InvalidArgumentException(`
`  'Status inválido.');`
`}`
	`if new_status not in STATUSES:`
	`  raise ValueError("Status inválido.")`
	`...`
	`if pickup is None:`
	`  raise LookupError(`
	`  f"Coleta #{pickup_id} não encontrada")`

Python tem exceções nativas parecidas com as do PHP: `ValueError` (valor inválido) e `LookupError` (não encontrado). O `main.py` captura as duas e mostra a mensagem, sem traceback.

4.4 storage.py: o Repository do JSON

É o único lugar que abre o arquivo. Se amanhã o projeto trocar JSON por banco, só esse arquivo muda, a mesma ideia de trocar a implementação de um Repository.

Laravel / PHP	Este projeto / Python

`$json = Storage::get('pickups.json');`
`$data = json_decode($json, true);`
`Storage::put('pickups.json',`
`  json_encode($data, JSON_PRETTY_PRINT`
`  | JSON_UNESCAPED_UNICODE));`
	`with open(DATA_PATH, encoding="utf-8") as file:`
	`  pickups = json.load(file)`
	
	`with open(DATA_PATH, "w",`
	`  encoding="utf-8") as file:`
	`  json.dump(pickups, file, indent=2,`
	`  ensure_ascii=False)`

`with open(...) as file:` abre o arquivo e **fecha sozinho** ao sair do bloco. `json.load/dump` são o `json_decode/encode`, `indent=2` é o PRETTY_PRINT e `ensure_ascii=False` mantém acentos (UNESCAPED_UNICODE).

O `storage.py` também protege contra arquivo ruim: se não existe, está vazio, tem JSON inválido, é ilegível ou tem registros fora do formato, ele devolve `[]` e registra um aviso no log, em vez de quebrar. Isso é o que permite a primeira execução funcionar sem nenhum arquivo.

4.5 main.py: o controller que orquestra

O `main()` carrega os dados uma vez, entra em um loop e chama a função certa para cada opção. Não tem regra de negócio: só liga as peças.

```
pickups = load_pickups()                  # carrega uma vez ao iniciar
while True:
    show_menu()
    option = input("Escolha uma opção: ").strip()
    if option == "0":
        break
    elif option == "1":
        pickups = create_pickup(pickups)  # guarda a lista NOVA devolvida
    elif option == "2":
        cli.print_table(list_pickups(pickups))
    ...
```

O `if/elif/else` faz o papel do **match** do PHP ou do roteador. O `while True` mantém o programa vivo até o `break`. Tudo isso fica dentro de um `try/except (KeyboardInterrupt, EOFError)` para que **Ctrl+C** e **Ctrl+D** encerrem com uma mensagem amigável, sem stack trace.

5. map, filter e reduce: as Collections do Python

No Laravel você usa **$collection->filter()->map()->reduce()**. No Python puro existem as funções `filter()`, `map()` e `functools.reduce()`. A ideia é a mesma, mas a sintaxe é **invertida**: a função vem primeiro e a lista depois, em vez de encadear métodos.

Laravel Collection	Python	Onde aparece no projeto
`->filter(fn)`	`filter(fn, lista)`	`filter_by_status`, `filter_by_priority`
`->first(fn)`	`next(filter(fn, lista), None)`	`find_by_id` (para na primeira e devolve `None` se não achar)
`->map(fn)`	`map(fn, lista)`	`to_display_row` na tabela e a troca de status em `update_status`
`->reduce(fn, 0)`	`reduce(fn, lista, 0)`	soma de pacotes e contagens do `build_summary`
`->sortBy('id')`	`sorted(lista, key=lambda p: p["id"])`	`list_pickups`

Laravel Collection	Python	Onde aparece no projeto
`->max('id') ?? 0`	`max((p["id"] for p in lista), default=0)`	`next_id`

Laravel / PHP	Este projeto / Python

`$pending = $pickups`
`  ->filter(fn($p) =>`
`  $p['status'] === $status)`
`  ->values()`
`  ->all();`
	`pending = list(`
	`  filter(`
	`  lambda p: p["status"] == status,`
	`  pickups,`
	`  )`
	`)`

Atenção: `filter()` e `map()` devolvem um **iterador preguiçoso**, não uma lista. Por isso o `list(...)` em volta: sem ele você não consegue usar `len()` nem imprimir o resultado.

Laravel / PHP	Este projeto / Python

`$updated = $pickups->map(fn($p) =>`
`  $p['id'] === $id`
`  ? [...$p, 'status' => $new]`
`  : $p`
`);`
	`updated = list(map(`
	`  lambda p: {**p, "status": new_status}`
	`  if p["id"] == pickup_id else p,`
	`  pickups,`
	`))`

`{**p, "status": new_status}` copia o dict `p` e troca só o campo `status` (como o spread de array do PHP). As outras coletas passam intactas.

O resumo com reduce

O `reduce` percorre a lista carregando um **acumulador**. Para somar pacotes o acumulador é um número começando em 0. Para contar por prioridade, o acumulador é um dict que já nasce com todas as categorias em 0, e cada coleta devolve um dict novo com +1 na sua categoria. Assim, categorias sem nenhuma coleta aparecem com 0.

```
total_packages = reduce(lambda acc, p: acc + p["package_count"], pickups, 0)

by_priority = reduce(
    lambda acc, p: {**acc, p["priority"]: acc[p["priority"]] + 1},
    pickups,
    {"low": 0, "medium": 0, "high": 0},      # acumulador inicial
)
```

> **Equivalente no Laravel**
> `$pickups->sum('package_count')` e `$pickups->countBy('priority')`. O Laravel tem atalhos prontos; no Python puro o **reduce** mostra o mecanismo por trás deles.

6. Validação e tratamento de erros

O projeto tem três níveis de defesa, cada um no lugar certo:

Nível	Onde	O que faz	Analogia Laravel
1	`cli.py`	Repete a pergunta até a resposta ser válida (texto vazio, letra no lugar de número, opção inexistente).	**FormRequest** / validação de campos
2	`pickups.py`	`validate_pickup_data` levanta `ValueError` se algo inválido chegar (nome vazio, pacotes <= 0, prioridade fora da lista).	Regras no Service / Model
3	`storage.py`	`is_valid_pickup` descarta o arquivo se os registros estiverem fora do formato.	Casts e validação ao ler dados externos

Quando algo dá errado, as funções **levantam uma exceção** e quem chama decide o que mostrar. Quem fala com o usuário é sempre o `main.py`, pelo `cli.print_message`.

```
try:
    updated = update_status(pickups, pickup_id, new_status)
except (ValueError, LookupError) as error:   # captura as duas
    cli.print_message(str(error))             # mostra só a mensagem
    return pickups                            # e segue, sem traceback
```

6.1 Logs

O módulo `logging` é o **Log** do Python. Em `main.py` ele é configurado para gravar só em `filafacil.log`, como se fosse o **config/logging.php** apontando para um canal de arquivo. Por isso nada de log aparece no terminal do usuário.

Laravel / PHP	Este projeto / Python
`Log::info('Coleta cadastrada')`	`logging.info("Coleta cadastrada: id=%s", id)`
`Log::warning('...')`	`logging.warning('...')` (arquivo de dados inválido)
`Log::error('...')`	`logging.error('...', exc_info=True)` (falha ao gravar o arquivo)
Formato definido no canal do log	`format="%(asctime)s [%(levelname)s] %(message)s"`

7. Estado, persistência e JSON x banco

No Laravel, cada requisição começa do zero e lê do banco. Aqui o programa **fica vivo** enquanto o menu está aberto, então dá para manter os dados em memória:

- **Ao iniciar:** `load_pickups()` lê o JSON uma vez e guarda a lista na variável `pickups`.
- **A cada alteração:** a função de regra devolve uma lista nova, o `save_pickups()` grava, e o `main.py` passa a usar a lista nova.
- **Se a gravação falhar** (`OSError`): o usuário vê um aviso e a lista em memória **não muda**, para a tela não mostrar algo que não foi salvo.

O formato do arquivo é uma lista de objetos, uma coleta por objeto:

```
[
  {
    "id": 1,
    "customer_name": "Mercado Bom Preço",
    "address": "Rua das Flores, 120",
    "package_count": 5,
    "priority": "high",
    "status": "pending"
  }
]
```

7.1 JSON x banco de dados

Laravel / PHP	Este projeto / Python
Tabela com colunas tipadas e **migrations**	Arquivo JSON sem schema: quem garante o formato é o código (`is_valid_pickup`)
id autoincrement do banco	`next_id()`: maior id + 1, calculado em Python
Consultas com **where()** no banco	Carrega tudo em memória e filtra com `filter()`
Transações e acesso concorrente	Nenhum: grava o arquivo inteiro de cada vez, só serve para um usuário por vez
Escala a milhões de linhas	Bom para estudo e poucos milhares de registros

> **Para ir além**
> A versão com banco deste mesmo sistema está na pasta **simple-database-manegement**, com SQLite e SQLAlchemy (parecido com Eloquent) e arquitetura Controller / Service / Repository. Comparar os dois mostra o que o banco resolve.

8. Como rodar e testar

Laravel / PHP	Este projeto / Python
`composer install`	Nada para instalar: só a biblioteca padrão
`php artisan app:pickups`	`python main.py` (ou `python3 main.py`)
PHP 8.x instalado	Python **3.11 ou superior**
`vendor/` (dependências)	`.venv/` (ambiente virtual), não necessário neste projeto
Escolher a versão do PHP (phpenv, Herd, Sail)	`uv`: `uv run --no-project --python 3.11 python main.py`
`php artisan test`	Este projeto não tem testes automatizados; os testes foram manuais (uma tabela de 15 casos)
`tail -f storage/logs/laravel.log`	`tail -f filafacil.log`

> **Atenção ao Python do seu sistema**
> O `python3` padrão da sua máquina é a versão 3.8, e o projeto usa recursos do 3.9 em diante (como `list[dict]` e `str.removeprefix`) e do 3.10 (`dict | None`). Por isso rode com o **uv** pedindo a 3.11. Se rodar com o 3.8 vai aparecer `TypeError: 'type' object is not subscriptable`.

Para zerar os dados e começar de novo, é como um **migrate:fresh**: apague os arquivos gerados.

```
rm -f pickups.json filafacil.log
```

9. Cola rápida

Eu quero...	No Laravel eu faria...	Neste projeto faço...
Adicionar um campo na coleta	Migration + **$fillable** + validação	Incluir no dict em `register_pickup`, validar em `validate_pickup_data` e mostrar em `to_display_row`
Adicionar um novo status	Novo case no Enum	Incluir em `STATUSES` e em `STATUS_LABELS` no `constants.py`
Nova opção no menu	Nova rota + método no controller	Novo `elif` no `main.py` chamando uma função nova
Nova regra de negócio	Método no Service	Nova função pura em `pickups.py` (sem `print` nem `input`)
Mudar onde os dados ficam	Trocar o Repository	Mudar só o `storage.py`
Depurar com **dd()**	`dd($x)`	`print(x)` temporário, ou `logging.info()` para ficar no log

Exercícios sugeridos

- Adicione o status **canceled** (Cancelada) só mexendo no `constants.py` e veja o menu atualizar sozinho. Por que não foi preciso mudar mais nada?
- Adicione um campo **notes** (observações) opcional à coleta. Em quais arquivos você mexeria?
- Escreva uma função `filter_by_customer(pickups, text)` em `pickups.py` usando `filter` e compare com `->filter()` do Laravel.
- Rode o programa, cadastre uma coleta, abra o `pickups.json` num editor, mude o status para `"xyz"` e rode de novo. O que acontece e por quê?